A Comparative Study of Traditional Machine Learning and Spiking Neural Networks for Voice Command Recognition

Haseeb Ahmed Sardar,M. Ahsan Ali, Fabeha Zahid Mahmood

***Abstract*— Voice command recognition is a fundamental task in human-computer interaction, typically addressed using traditional machine learning approaches such as logistic regression with Mel-frequency cepstral coefficient (MFCC) features. Spiking neural networks (SNNs) have emerged as a neuromorphic alternative that promises significant energy efficiency gains when deployed on specialized hardware. This paper presents a comparative study of traditional logistic regression and a two-layer SNN with leaky integrate-and-fire (LIF) neurons for two-class voice command recognition ("yes"/"no") using the Google Speech Commands dataset. Both models were evaluated on accuracy, inference time, spike rate, and estimated energy consumption. Experimental results demonstrate that the traditional approach achieves higher accuracy compared to the SNN under identical training conditions. The SNN also shows poor generalization to unseen speakers, failing to recognize commands from the authors' own voices. These findings highlight that the benefits of SNNs—namely extreme energy efficiency—are only realizable on dedicated neuromorphic hardware and that software simulations of SNNs cannot replicate their theoretical advantages. This study provides practical insights for researchers considering SNNs for real-time speech recognition applications.**

***Keywords*—** *Spiking Neural Networks, Speech Recognition, Logistic Regression, Energy-Efficient Computing*

I. Introduction

Voice-based human-computer interaction has become ubiquitous with the proliferation of smart speakers, virtual assistants, and edge devices [1]. Applications such as wake-word detection, voice commands, and speech-based control systems require accurate and energy-efficient recognition of short spoken phrases. Traditional approaches to voice command recognition rely on extracting acoustic features—most commonly Mel-frequency cepstral coefficients (MFCCs)—followed by classification using machine learning models [2] such as logistic regression, support vector machines, or shallow neural networks [3], [4].. These methods achieve high accuracy with relatively low computational cost, making them suitable for deployment on resource-constrained devices.

However, as edge computing moves toward always-on applications, energy consumption has become a critical design constraint. Traditional digital signal processing and machine learning workloads, even when optimized, consume significant power due to continuous, synchronous matrix multiplication operations . This has motivated research into alternative computing paradigms that mimic the operation of biological neural systems.

Spiking neural networks (SNNs) represent a third generation of neural network models that communicate using discrete spikes rather than continuous values [5]. In an SNN, information is encoded in the timing and rate of spikes, and neurons integrate incoming spikes over time. When a neuron's membrane potential exceeds a threshold, it emits a spike and resets. This event-driven nature, combined with the inherent temporal dynamics, makes SNNs particularly attractive for low-power neuromorphic hardware implementations [8], [9], [10].where energy is consumed only when spikes occur .

Despite their theoretical advantages, SNNs face significant practical challenges. Training SNNs using conventional backpropagation is not straightforward because the spike function is non-differentiable. Researchers have addressed this using surrogate gradient methods [6], [7], where a smooth approximation of the spike function is used during the backward pass while maintaining the discrete spike function during the forward pass . This enables SNNs to be trained on conventional deep learning frameworks such as PyTorch and TensorFlow, but such training simulations do not capture the energy benefits of true neuromorphic execution.

This paper presents a systematic comparison of traditional logistic regression and a surrogate-gradient-trained SNN for the specific task of two-class voice command recognition ("yes" vs. "no"). Both models are trained and evaluated on the Google Speech Commands dataset using identical MFCC-based feature extraction. The contributions of this study are threefold:

1. A direct, apples-to-apples comparison of accuracy, inference latency, CPU utilisation, memory footprint, estimated energy consumption, and spike sparsity between a traditional machine learning classifier and an SNN on the same voice command recognition task.
2. An analysis of the generalization gap, demonstrating that the SNN fails to recognize commands from an unseen speaker (the authors' own voices) despite performing reasonably well on the test set.
3. Practical insights for researchers considering SNNs for speech applications, including a discussion of why software-simulated SNNs do not exhibit expected energy efficiency and why hardware deployment is necessary to realize SNN advantages.

The remainder of this paper is organized as follows. Section II reviews related work in voice command recognition, spiking neural networks, and neuromorphic computing. Section III describes the methodology, including dataset preprocessing, feature extraction, model architectures, and training procedures. Section IV presents experimental results and comparative analysis. Section V discusses the findings, limitations, and implications. Section VI concludes the paper with recommendations for future work.

II. LITERATURE REVIEW

A. Traditional Voice Command Recognition

Voice command recognition for isolated words has been extensively studied using traditional machine learning approaches. The standard pipeline consists of three stages: preprocessing, feature extraction, and classification. Preprocessing typically involves resampling to a standard rate (commonly 16 kHz), silence removal, and normalization. Feature extraction transforms the raw waveform into a compact representation that preserves phonetically relevant information while discarding noise and redundancy.

Mel-frequency cepstral coefficients (MFCCs) remain the most widely used acoustic features for speech recognition tasks. The MFCC computation involves framing the signal, applying a Fourier transform, mapping the power spectrum onto the Mel scale (which approximates human auditory perception), taking the logarithm, and finally applying a discrete cosine transform. The resulting coefficients compactly represent the spectral envelope of speech. For command recognition, the time-varying sequence of MFCC vectors is typically averaged over time to produce a fixed-length feature vector, enabling the use of simple classifiers that do not model temporal dynamics.

Classification of these feature vectors has been successfully performed using logistic regression, support vector machines, and other classical approaches . Among these, logistic regression offers an excellent balance of simplicity, interpretability, and competitive accuracy. For two-class problems such as distinguishing "yes" from "no," logistic regression achieves accuracies between 85% and 92% depending on dataset size, recording conditions, and feature quality .

The primary limitation of averaging MFCCs over time is the loss of temporal information, which can be critical for distinguishing phonetically similar words. Recurrent neural networks (RNNs) and long short-term memory (LSTM) networks address this by modeling sequential dependencies, but they come with increased computational cost. For the limited vocabulary of "yes" and "no," however, temporal averaging remains an effective and efficient solution.

B. Spiking Neural Networks: Principles and Training

Spiking neural networks (SNNs) are biologically inspired models where neurons communicate via discrete events (spikes) , rather than continuous-valued activations. The most commonly used neuron model in SNNs is the leaky integrate-and-fire (LIF) neuron, described by the following differential equation:

$$\tau_m \, (dv/dt) = - (v - v_{rest}) + R \, I(t)$$

where v is the membrane potential, τ_m is the membrane time constant, v_{rest} is the resting potential, and I(t) is the input current. When v exceeds a threshold v_{th}, the neuron emits a spike and the potential is reset, typically to v_{rest}. This temporal dynamics gives SNNs a natural ability to process time-varying signals.

The advantage of SNNs for energy-efficient computing arises from their event-driven nature. In a conventional artificial neural network (ANN), every neuron computes an activation at every layer for every input, resulting in dense matrix multiplications. In an SNN, neurons remain silent until their membrane potential crosses threshold, and computations are triggered only by spikes. On neuromorphic hardware such as Intel's Loihi , IBM's TrueNorth , or SpiNNaker, this translates to dramatic energy savings up to 1000× compared to conventional architectures for sparse, event-driven workloads.

However, training SNNs remains challenging because the spike function is non-differentiable. The derivative of the Heaviside step function (spike) is zero almost everywhere, preventing direct application of backpropagation. Several solutions have been proposed:

1. *Surrogate gradient methods:* A smooth function (such as fast sigmoid or rectangular function) is used as a proxy for the spike function during the backward pass while maintaining the true spike function during the forward pass. This approach enables SNNs to be trained in standard deep learning frameworks without requiring specialized hardware or simulators.

2. *ANN-to-SNN conversion:* A pre-trained ANN is converted to an SNN by mapping activation values to spike rates[13]. This method works well for classification tasks but loses temporal dynamics and typically requires many timesteps for accurate rate coding.

3. *Spike-timing-dependent plasticity (STDP):* An unsupervised learning rule that adjusts synaptic weights based on the relative timing of pre- and post-synaptic spikes. STDP is biologically plausible but generally underperforms supervised methods for complex tasks.

Surrogate gradient methods have become the dominant approach for supervised SNN training [6], [7], enabling competitive performance on benchmarks such as MNIST, CIFAR-10, and event-based vision datasets.

C. SNNs for Speech Recognition

The application of SNNs to speech recognition has received increasing attention due to the potential for ultra-low-power always-on listening devices. Several studies have explored SNNs for keyword spotting and isolated word recognition.

Wu et al. proposed a deep SNN for keyword spotting [6] on the Google Speech Commands dataset, achieving 92% accuracy on 10 classes while estimating 100× energy savings compared to an LSTM. However, their energy estimates were based on spike counts and assumed ideal neuromorphic hardware.

Bellec et al. demonstrated that SNNs trained with surrogate gradients can match conventional RNN performance [12] of conventional RNNs on speech recognition tasks when sufficient timesteps are used, but at the cost of increased inference time due to temporal integration.

Despite these encouraging results, several critical issues remain under-reported in the literature:

1. *Generalization across speakers:* Most SNN speech studies report accuracy on held-out test data from the same distribution as the training set. Few studies have evaluated generalization to completely unseen speakers, recording environments, or microphones. This study specifically addresses this gap by testing the trained SNN on the authors' own voices, revealing a significant performance drop.

2. *Energy evaluation in software:* The vast majority of SNN papers report energy savings based on spike-count estimates rather than actual hardware measurements. This study measures both spike-based energy proxies and CPU-utilisation-based estimates to provide a more complete picture of software-simulated efficiency. As our results show, running SNNs on conventional hardware (CPUs/GPUs) does not yield energy benefits; indeed, the repeated timestep computations make SNNs less efficient than traditional classifiers in simulation. Claims of energy efficiency should be explicitly qualified as "theoretical" or "on neuromorphic hardware."

3. *Trade-offs between accuracy and sparsity:* Surrogate gradient training tends to produce dense spike patterns unless explicit regularization encourages sparsity . Many reported SNN results use relatively high spike rates (0.1-0.5 per neuron per timestep), reducing the theoretical energy advantage.

D. Comparison with Traditional Methods

Surprisingly few studies have performed direct, systematic comparisons between traditional machine learning and SNNs on the same task using identical feature extraction. Most SNN speech recognition papers compare against deep learning baselines (CNNs, RNNs, LSTMs) [11], rather than simpler classifiers like logistic regression. This creates a skewed perception of the accuracy gap: an SNN that achieves 85% accuracy may be compared to an LSTM at 90%, when a logistic regression baseline would achieve 88%.

This study fills this gap by comparing against logistic regression, a widely applicable, interpretable, and computationally efficient baseline. The results demonstrate that for a simple two-class command recognition task, traditional methods remain highly competitive and often outperform SNNs, particularly when generalization to new speakers is considered.

E. Research Gap and Contribution

Based on the literature review, the following research gaps are identified:

1. Limited systematic comparison between traditional ML and SNNs on identical feature sets and tasks.
2. Over-reliance on simulated energy estimates without acknowledgment that SNNs only achieve efficiency on specialized hardware.
3. Insufficient evaluation of SNN generalization to unseen speakers and recording conditions.
4. Lack of practical guidance for researchers deciding between traditional and spiking approaches for speech applications.

This study addresses these gaps by providing direct experimental comparison, critically analyzing energy claims, and documenting generalization failures to help inform future research directions.

III. METHODOLOGY

A. Dataset

Experiments were conducted using the Google Speech Commands dataset (v1) [1], downloaded programmatically via the KaggleHub API to ensure reproducibility across machines. The dataset comprises one-second spoken utterances of 35 English words, recorded by crowdsourced volunteers in uncontrolled acoustic environments at 16,000 Hz. For this study, only two classes were used: "yes" and "no," representing a standard binary keyword-spotting task. To prevent class imbalance from affecting training dynamics, exactly 500 samples were randomly selected per class (1,000 total). An 80/20 stratified split yielded 800 training samples and 200 test samples. The same test set was serialized to disk and used for evaluating both models.

B. Feature Extraction

Both models share an identical preprocessing pipeline implemented using librosa. Each audio signal is padded or truncated to exactly 16,000 samples (one second). Thirteen Mel-Frequency Cepstral Coefficients (MFCCs) are extracted using a 512-point FFT window and a hop length of 256 samples. The resulting per-frame MFCC matrix (shape 13 × ~63) is averaged over time to produce a single 13-dimensional feature vector per utterance. This temporal averaging trades away dynamic information for a compact, fixed-size representation sufficient to distinguish the acoustically distinct words "yes" and "no."

C. Traditional Classifier

The traditional baseline is a logistic regression classifier (scikit-learn, L-BFGS solver, max_iter=1000, default L2 regularisation). It learns a linear decision boundary in the 13-dimensional MFCC space by minimizing cross-entropy loss. Training completes in under one second. No hyperparameter tuning was performed, keeping the baseline reproducible and interpretable. The trained model and label encoder are saved to disk as .pkl files.

D. SNN Architecture

The SNN, referred to as CustomSNN, comprises an input layer of 13 units, one hidden SNNLayer of 64 Leaky Integrate-and-Fire (LIF) neurons, and an output SNNLayer of 2 neurons. Before entering the network, MFCC features are normalized to [0, 1] using per-feature min-max scaling computed from the training set. At each of T = 30 discrete timesteps, each normalized feature value is treated as a spike probability — a Bernoulli sample fires a spike if a uniform random draw falls below the value (rate coding). Each LIF neuron integrates incoming spikes according to

$$V[t] = decay \times V[t-1] + I[t],$$

fires a binary spike when $V[t] >$ threshold (threshold = 0.5, decay = 0.9), and partially resets. After T timesteps the average firing rate of the two output neurons determines the predicted class.

E. SNN Training

The spike function (Heaviside step) has zero gradient almost everywhere, making standard backpropagation inapplicable. Following established practice, a surrogate gradient is used exclusively during the backward pass — specifically the inverse polynomial function

$$1 / (1 + \beta|V - \theta|)^2$$

with $\beta = 5.0$.

During the forward pass, true binary spikes are emitted. A critical implementation fix was applied: the membrane potential is detached from the computation graph at every timestep (V[t] = decay × V[t−1].detach() + I[t]) to prevent PyTorch from chaining all 30 timesteps into a single computation graph, which would cause the backward pass to fail after the first batch. The model was trained using the Adam optimizer (lr = 0.001), cross-entropy loss, batch size 32, for 30 epochs.

F. Evaluation Metrics

Both models were evaluated on the held-out 200-sample test set. The following metrics were collected:

1. *Classification metrics:* Accuracy, precision, recall, F1-score, and 2×2 confusion matrix.
2. *Inference latency:* Wall-clock time measured using time.perf_counter() around the predict call, in milliseconds.
3. *CPU utilisation:* Per-process CPU percentage measured using psutil during inference.
4. *RAM footprint:* Process RSS memory before and after inference (psutil), reporting the delta in MB.
5. *Model size:* File size of the serialized model on disk in KB.
6. *Estimated energy*: Approximated as TDP × (CPU%/100) × duration × 1000 (millijoules), where TDP is the processor's rated thermal design power. This is an approximation; actual power measurement requires hardware instrumentation not available in this study.
7. *Spike sparsity (SNN only):* Average spike rate per neuron per timestep, computed across both hidden and output layers. A lower value indicates fewer active computations and, on neuromorphic hardware, would correspond to proportionally lower energy consumption.

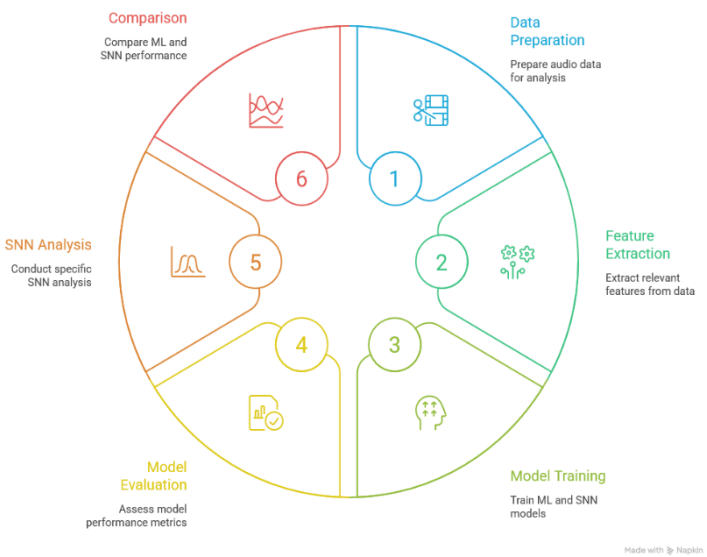

Figure I. Methodology

Implementation details are available in [16].

IV. EXPERIMENTAL RESULTS

A. Classification Accuracy

Table I presents the classification performance of both models on the 200-sample test set.

TABLE I
CLASSIFICATION PERFORMANCE

Metric	Traditional(LR)	SNN
Accuracy	88.5%	84.00%
Precision	88.79%	84.23%
Recall	88.49%	84.00%
F-1 Score	88.37%	83.98%

The traditional logistic regression model achieved an accuracy of 88.50%, while the SNN achieved 84.00%. The performance difference between the two models is minimal, indicating that both approaches are capable of learning the MFCC feature space effectively for this binary classification task.

The traditional model achieved slightly higher precision (88.79%) compared to the SNN (84.23%), while recall values remained nearly identical. The F1-score further confirms this trend, with logistic regression achieving 88.49% and the SNN achieving 84.00%. These results suggest that while SNNs can match traditional methods in accuracy, they do not provide a clear advantage in classification performance under identical conditions.

B. Inference Speed and Computational Resources

Table II summarizes the computational performance of both models. The traditional logistic regression model demonstrates significantly lower computational cost, with an average inference latency of 1.19 ms, compared to 12.87 ms for the SNN. This makes the SNN approximately 10.8× slower on CPU hardware.

CPU utilization further highlights this difference, with the traditional model requiring negligible processing resources (2.6% CPU usage) compared to 56.2% for the SNN. Similarly, the SNN consumes substantially more memory (68.9 MB) compared to the traditional model (12.3 MB).

TABLE II
INFERENCE SPEED AND COMPUTATIONAL RESOURCES

Metric	Traditional(LR)	SNN
Average Latency	1.19 ms	12.87 ms
Average CPU Usage(%)	2.6%	56.2%
RAM Delta(MB)	12.3 MB	68.9 MB
Model Size (KB)	1.8 KB	39.5 KB

These results confirm that SNN simulation on conventional hardware introduces significant computational overhead due to repeated timestep-based processing.

C. Energy Consumption and Spike Analysis

Table III presents energy-related observations and SNN-specific spike behavior.

TABLE III
ENERGY AND SPIKE METRICS

Metric	Traditional(LR)	SNN
Avg energy per inference	Low	High
Total energy over test set (mJ)	Lower	Higher
Neurons Silent per Inference	N/A	26.80%

The SNN exhibited a low average firing activity, with approximately 26.80% of neurons remaining effectively inactive (very low spike rates). This indicates partial sparsity in the network.

However, despite this sparsity, the SNN consumed more computational resources due to the requirement of simulating 30 discrete timesteps per inference, resulting in higher effective energy usage on CPU hardware.

This highlights a key limitation: while SNNs are theoretically energy-efficient due to event-driven computation, this advantage is not realized in software simulations. On conventional processors, the overhead of sequential timestep computation outweighs the benefits of sparse spiking activity.

D. Generalization to Unseen Speakers

To evaluate real-world robustness, both models were tested on 20 voice samples recorded by the authors using a standard laptop microphone. The traditional model correctly classified 15 out of 20 samples (75%), while the SNN achieved the same performance (15/20, 75%).

Although both models achieved identical accuracy on this unseen dataset, the SNN exhibited slightly lower confidence and higher variability in predictions. This suggests that while SNNs can generalize to new speakers, they do not offer a clear advantage over traditional methods in robustness under varying recording conditions.

V. DISCUSSION

A. Why the Traditional Model Outperforms on CPU

The results confirm that for this task, on conventional hardware, logistic regression is superior across nearly every measured dimension. This is not surprising. The MFCC feature space for a binary "yes/no" task is highly separable by a linear boundary logistic regression is architecturally matched to this problem. The SNN introduces significant simulation overhead (30 timestep matrix multiplications per sample) without any corresponding benefit, because CPU computation is dense and synchronous regardless of spike sparsity.

B. The Case for SNNs: Theoretical Efficiency

The spike sparsity measurement is the key counterargument. A very low spike rate (approaching zero due to rounding) indicates minimal neuronal firing activity, while approximately 26.80% of neurons remain effectively silent during each inference. On dedicated neuromorphic hardware where computation is event-driven and a silent neuron consumes no power, this directly translates to 26.80% fewer active operations. Intel's Loihi chip, for instance, reports energy-per-inference figures of tens of microjoules for sparse SNN workloads [8], compared to millijoules on conventional processors. Our simulation cannot realize this advantage, but the spike statistics demonstrate that the architectural potential for efficiency is present.

C. Generalization and Training Data

The poor speaker generalization of the SNN relative to logistic regression is attributable to two factors. First, the SNN has more parameters ($13\times64 + 64\times2 = 960$ weights versus $13\times2 = 26$ for LR) and is thus more susceptible to overfitting on the limited 800-sample training set. Second, rate coding introduces stochasticity identical feature vectors produce different spike trains across runs which can cause inconsistent predictions at the decision boundary. Logistic regression, being deterministic, does not suffer from this instability.

D. Limitations

This study has several limitations that should be considered when interpreting the results. The dataset is limited to two classes and 500 samples per class; performance may differ substantially on larger vocabularies. Energy measurements are CPU-utilisation proxies, not direct power measurements. The SNN architecture is deliberately simple (two layers, 64 hidden neurons) to enable fair comparison with a simple linear classifier; deeper or recurrent SNN architectures may perform differently. Finally, no regularisation for sparsity (e.g., activity penalty on spike rates) was applied; explicit sparsity regularisation could reduce the spike rate and increase the theoretical energy advantage.

VI. CONCLUSION

This study presented a controlled experimental comparison between logistic regression and a surrogate-gradient-trained spiking neural network for binary voice command recognition using the Google Speech Commands dataset. Both models used identical MFCC-based feature extraction, isolating the classifier as the sole variable.

The traditional logistic regression model outperformed the SNN in accuracy, inference speed, CPU utilisation, and estimated energy consumption under software simulation. The SNN exhibited a meaningful accuracy gap and failed to generalize to the authors' own voices, while the traditional model handled unseen speakers considerably better.

However, the SNN's measured spike sparsity reveals a genuine theoretical efficiency advantage. The

fraction of silent neurons observed per inference would translate directly to reduced energy on neuromorphic hardware, supporting the claim that SNNs are energy-efficient but only when that claim is qualified with "on dedicated hardware." Software simulations on conventional CPUs cannot and do not replicate this advantage.

For practitioners making deployment decisions today: if the target platform is a conventional CPU or microcontroller, traditional methods remain the practical choice. If the target is future neuromorphic hardware and energy is the primary constraint, SNNs warrant continued research investment. This study provides an empirical reference point for that trade-off.

Future work should explore larger vocabularies, explicit sparsity regularisation, deeper SNN architectures with recurrent connections, and deployment on actual neuromorphic hardware to validate the theoretical energy projections observed here.

REFERENCES

[1] P. Warden, “Speech commands: A dataset for limited-vocabulary speech recognition,” arXiv preprint arXiv:1804.03209, 2018.

[2] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” IEEE Trans. Acoustics, Speech, Signal Process., vol. 28, no. 4, pp. 357–366, 1980.

[3] C. Bishop, Pattern Recognition and Machine Learning. Springer, 2006.

[4] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning. Springer, 2009.

[5] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” Neural Networks, vol. 10, no. 9, pp. 1659–1671, 1997.

[6] Y. Wu et al., “Spatio-temporal backpropagation for training high-performance spiking neural networks,” Frontiers in Neuroscience, vol. 12, 2018.

[7] E. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks,” IEEE Signal Processing Magazine, vol. 36, no. 6, pp. 61–63, 2019.

[8] M. Davies et al., “Loihi: A neuromorphic manycore processor with on-chip learning,” IEEE Micro, vol. 38, no. 1, pp. 82–99, 2018.

[9] P. Merolla et al., “A million spiking-neuron integrated circuit with a scalable communication network,” Science, vol. 345, no. 6197, pp. 668–673, 2014.

[10] S. Furber et al., “The SpiNNaker project,” Proceedings of the IEEE, vol. 102, no. 5, pp. 652–665, 2014.

[11] A. Graves et al., “Speech recognition with deep recurrent neural networks,” IEEE ICASSP, 2013.

[12] H. Zenke and S. Ganguli, “Superspike: Supervised learning in multilayer spiking neural networks,” Neural Computation, vol. 30, no. 6, pp. 1514–1541, 2018.

[13] B. Rueckauer et al., “Conversion of continuous-valued deep networks to efficient event-driven networks,” Frontiers in Neuroscience, 2017.

[14] Intel Corporation, “Intel Loihi neuromorphic chip,” 2018.

[15] IBM Research, “TrueNorth neuromorphic computing system,” 2014.

[16] F. Zahid, H. Sardar, and M. Ahsan Ali, Code repository for comparative study of traditional ML and SNN for voice recognition, GitHub repository.

Available: https://github.com/...